+ Lecture No. : +

■ **Theory Content to be taught :**

1. Unit 5 : Sorting & Hashing.
2. - sorting. - Introduction.
3. ① Bubble sort.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

● sorting :-

Sorting is one of the operations which we can perform on data structure in order to arrange data elements in ascending and descending order.

The sorting methods mainly categorized into two broad categories :

① Internal storage sorting : If the file to be sorted is small enough, so that the

sort can be carried out in the main memory, this type of sorting is known as Internal storage sorting.

② External storage sorting :-

If the file to be sorted is very large, requiring a large amount of physical memory, it may be sorted in secondary memory (such as Hard disk). This type of sorting is known as External storage sorting.

[redacted]

① Bubble Sort :-

Bubble sort is a simple sorting algorithm. This sorting algorithm is comparision based algorithm in which each pair of adjacent elements is compared and the elements are swapped if they are not in order.

Bubble sort start by comparing first element with the second element, if first element is greater than second element then it will swap both the elements and more on to compare second element with third element and so on.

this algorithm is not suitable for large data sets on it's average and worst

case complexity is $O(n^2)$, where 'n' is the number of items.

● **Algorithm :-**

Let a[max] is an array and 'n' is the size of an array,

Step 1 : Repeat for i=1 to n
step 2 : Repeat for j=0 to n-i.
step 3 : if (a[j] > a[j+1])
step 4 : exchange a[j] & a[j+1].
step 5 : print sorted array.

Example :- sort following array in ascending order using Bubble sort. 5, 9, 6, 2, 8, 1.

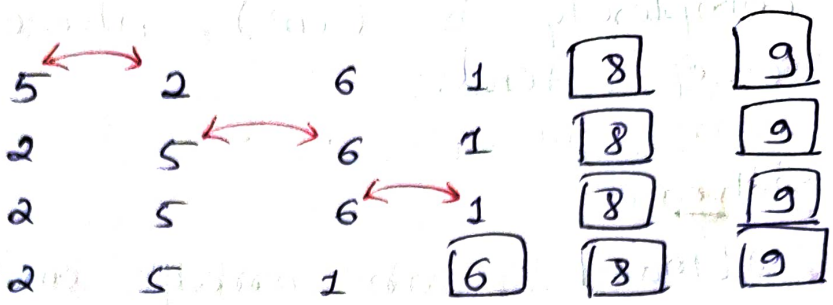
pass 1 :

5	9	6	2	8	1
5	9	6	2	8	1
5	6	9	2	8	1
5	6	2	9	8	1
5	6	2	8	9	1
5	6	2	8	1	9

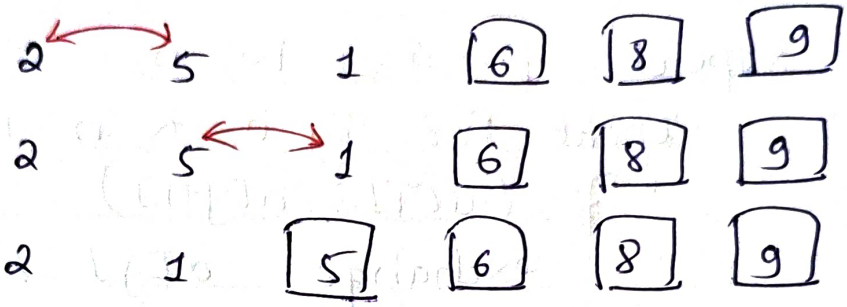
pass 2 :

5	6	2	8	1	9
5	6	2	8	1	9
5	2	6	8	1	9
5	2	6	8	1	9
5	2	6	1	8	9

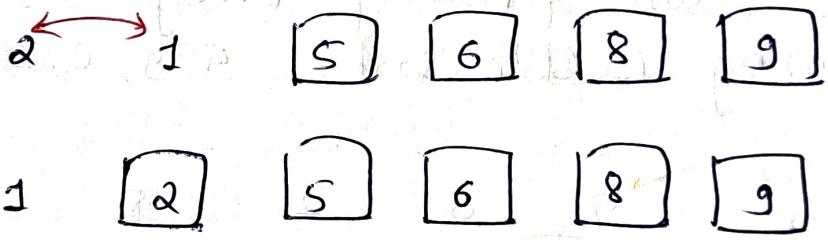
Pass 3:



Pass 4:



Pass 5:



Sorted Array :-

1 2 5 6 8 9.

● C program to implement Bubble sort :-

```
#include<stdio.h>
void main()
{
    int a[30], n, i, j, temp;
    clrscr();
```

```
printf ("enter the size of an array");
scanf ("%d", &n);
printf ("enter %d array elements");
for (i=0; i<n; i++)
 {
    scanf ("%d", &a[i]);
 }
for ( i=1; i<n ; i++)
 {
   for (j=0; j<n-i ; j++)
   {  if (a[j] > a[j+1])
      {
        temp = a[j];
        a[j] = a[j+1];
        a[j+1] = temp;
      }
   }
 }
printf (" sorted elements are —");
for (i=0; i<n; i++)
{
    printf ("%d\t", a[i]);
}
getch();
}.
```

- Advantages:-

1. Very simple to understand. - Easy logic : Compare and swap adjacent elements.
2. Easy to implement - Requires minimal code.
3. In-place sorting. - Uses O(1) extra space. No additional memory required.

- Disadvantages:-

1. Very slow for large data sets. : not suitable for large data set. Because the average & worst case (Time complexity) is $O(n^2)$.
2. Too many comparisions. : Every pass compares adjacent elements repeatedly.
3. Too many swaps :- Frequent swapping increases execution time. Especially inefficient when data is large.

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Sorting : ② Selection sort.
2. ③ Insertion sort.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

② Selection Sort :-

Selection sort algorithm is used to ~~sort~~ arrange a list of elements in a particular order (ascending or descending).

In selection sort, the first element in the list is selected and it is compared repeatedly with remaining all the elements in the list. If any element is smaller than the selected element (for ascending order)

then both are swapped. Then we select the element at second position in the list and it is compared with remaining elements in the list. If any element is smaller than the selected element, then both are swapped. This procedure is repeated till the entire list is sorted.

Algorithm :-

step 1: Select the first position element of the list.

step 2: Compare the selected element with all other elements in the list.

step 3: For every comparision, if any element is smaller than selected element, then these two are swapped.

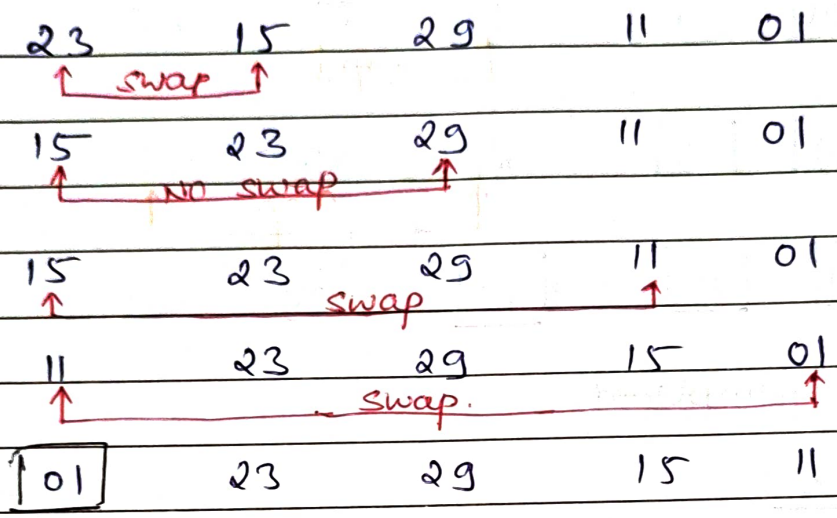
step 4: Repeat the same procedure with next position in the list till the entire list is sorted.

'C' program logic :-

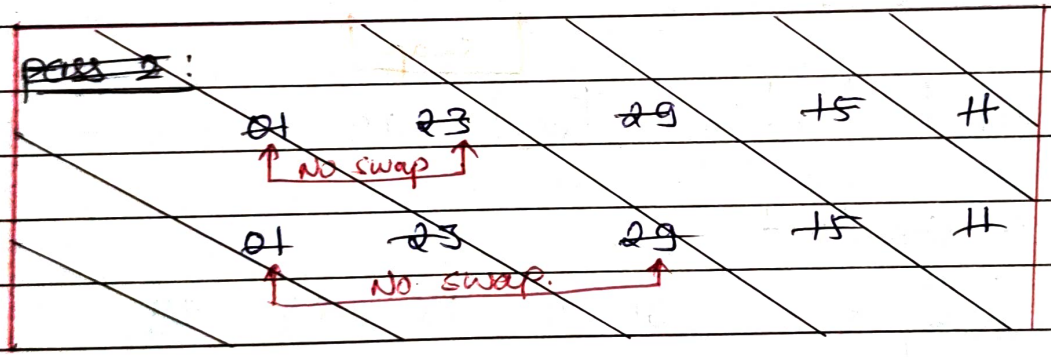
```
for (i=0 ; i<size ; i++)
{
  for (j= i+1 ; j<size ; j++)
  {  if (a[i] > a[j])
     {
        temp = a[i];
        a[i] = a[j];
        a[j] = temp;
     }
  }
}
```

Example: Array a[] = { 23, 15, 29, 11, 01 }.

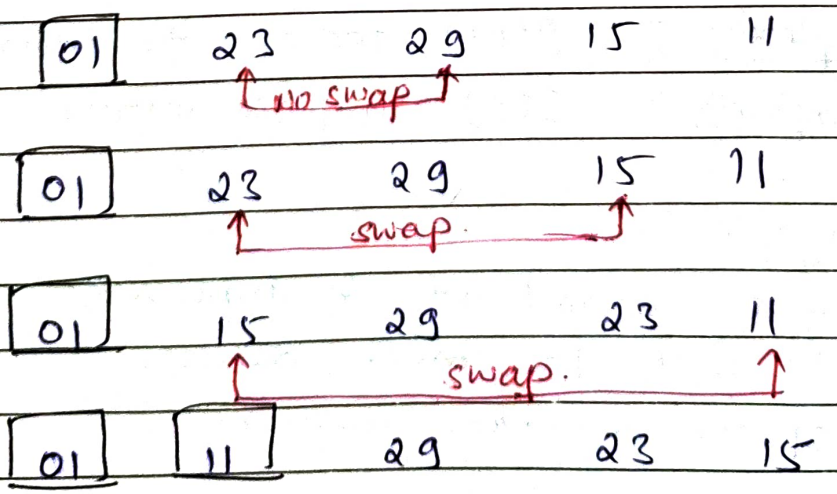
Pass 1:



pass 1 Completed.

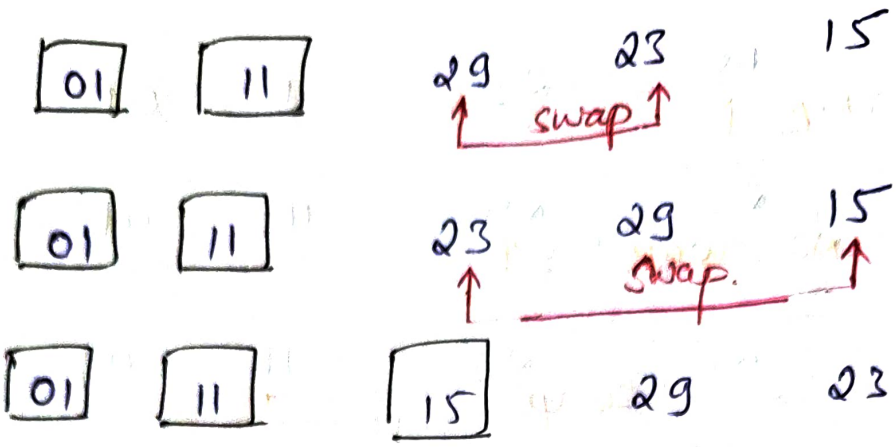


pass 2:



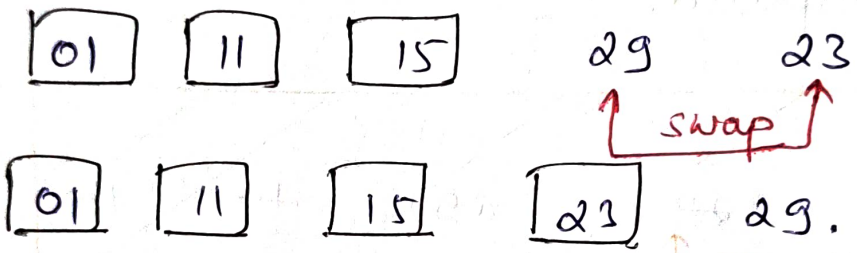
pass 2 Completed.

Pass 3:



pass 3 Completed.

Pass 4:



pass 4 Completed.

sorted array is = 01 11 15 23 29.

Time Complexity :- $O(n^2)$ for all the cases.

Space Complexity :- $O(1)$ in-place sorting.

- Advantages :-
  1) Easy to understand & implement.
  2) Works well for small datasets.
  3) Requires no extra memory.
- Disadvantages :-
  1) very slow for large datasets.
  2) Always $O(n^2)$, even for sorted data.

(3) Insertion Sort :-

Insertion sort algorithm arranges a list of elements in a particular order.

In insertion sort algorithm, every iteration moves, an element from unsorted portion to sorted portion until all the elements are sorted in the list.

In this method the elements are inserted at their appropriate place. Hence, the name "Insertion sort".

- **Working Principle :-**

Insertion sort works as follows -

1. Assume the first element is already sorted.
2. select the next element (called the key).
3. Compare the key with the elements on its left.
4. shift all larger elements one position to the right.
5. Insert the key into its correct position.
6. Repeat the process until all elements are sorted.

- **Algorithm :-**

1. start from the second element.
2. store the current element as key.
3. Compare the key with the privious elements.
4. Shift larger elements one position to the right.

5. Insert the key into its correct position.
6. Repeat the process for all remaining elements.
7. Stop when the entire array is sorted.

Example :- Array a[] = {25, 12, 20, 15, 30}.

pass 1: key = 12.

Compare with left side i.e. 25.

25 > 12. ⇒ shift 25. ⇒ Insert 12.

12 25 20 15 30

pass 2: key = 20.

Compare with 25.

25 > 20, ⇒ shift 25.

Again compare key with 12.

20 > 12. No shift.

Insert after 12. ⇒ 12 20 25 15 30

pass 3: key = 15.

Compare with 25 ⇒ 25 > 15 ⇒ shift.

Compare with 20 ⇒ 20 > 15 ⇒ shift

Compare with 12 ⇒ 15 > 12 ⇒ No shift.

Insert after 12 ⇒ 12 15 20 25 30

pass 4: key = 30

Compare with 25 ⇒ 30 > 25 ⇒ No shift.

⇒ 12 15 20 25 30.

final sorted array :-

12 15 20 25 30.

'C' program Logic :-

```
for (i=1; i<n-1; i++)
{
    key = a[i];
    j = i-1;
    while (j>=0 && a[j] > key).
    {
        a[j+1] = a[j];
        j = j-1;
    }
    a[j+1] = key;
}.
```

- Time Complexity :- Best case = O(n).
  Average case & Worst case = $O(n^2)$.
- Space Complexity :- O(1).

- Advantages :-
  1. Easy to understand & simple to implement.
  2. Requires only O(1) extra memory.
  3. Efficient for small datasets.

- Disadvantages :-
  1. Slow for large datasets.
  2. Worst case time complexity is $O(n^2)$.
  3. Many element shifts are required.

+ Lecture No. : _____ +

■ **Theory Content to be taught :**

1. Sorting : (4) Quick sort.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

(4) Quick sort :-

Quick sort is highly efficient sorting algorithm and is based on partitioning of array of data into smaller arrays.

A large array is partitioned into two arrays one of which holds values smaller than the specified value, say "pivot", based on which the partition

is made and another array holds values greater than the pivot value.

Quick sort partitions an array and then calls itself recursively twice to sort the two resulting subarrays.

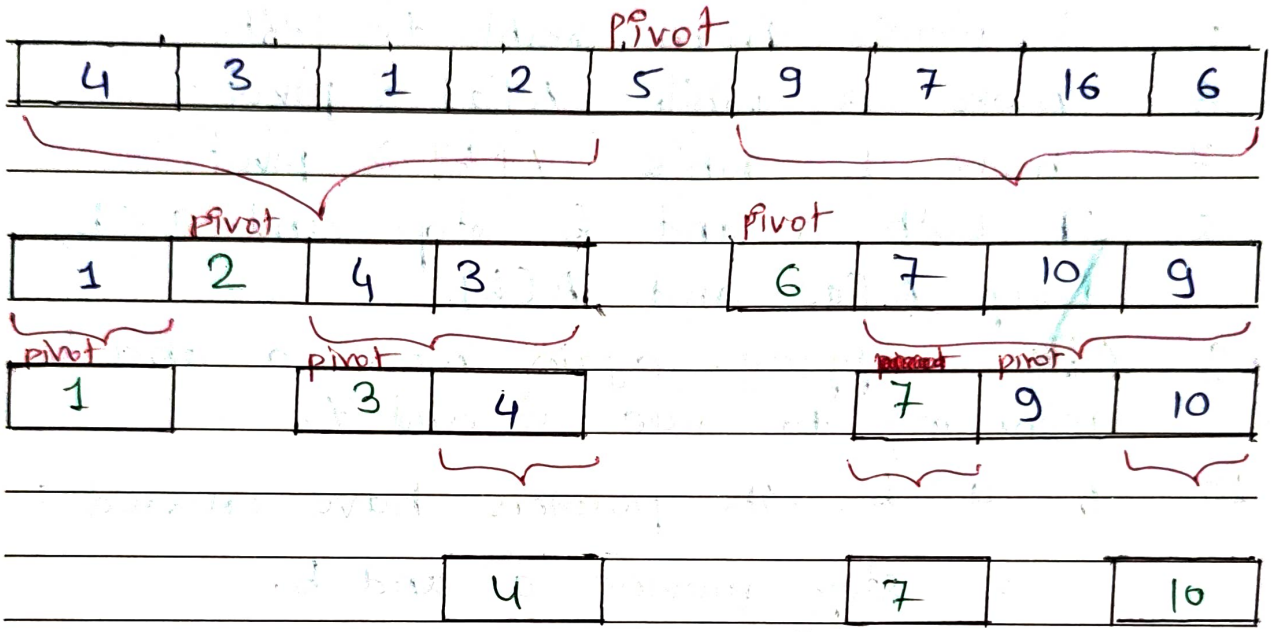
This algorithm is quite efficient for large sized data sets as its average and best case complexity are $O(n \log n)$ and worst case $O(n^2)$.

In this method, division is dynamically carried out. The three steps of Quick sort are as follows —

① Divide :- Split the array into two sub arrays that each element in the left sub array is less than or equal to the middle element. and each element in the right sub array is greater than the middle element. The splitting of the array into two sub arrays is based on 'pivot' element. All the elements that are less than pivot should be in the left sub-array and all the elements that are greater than pivot should be in the right sub array.

② Conquer :- Recursively sort the two sub arrays.

③ Combine :- Combine all the sorted elements in group to form a list of sorted elements.



Quick Sort :- first element as pivot :-

- ✓ pivot :- first element
- ✓ a :- second element
- ✓ b :- Last element.
- ✓ a moves left → right
- ✓ b moves right → left.

* Rules for pivot, a and b :-

1. pivot is always the first element of the current sub array.

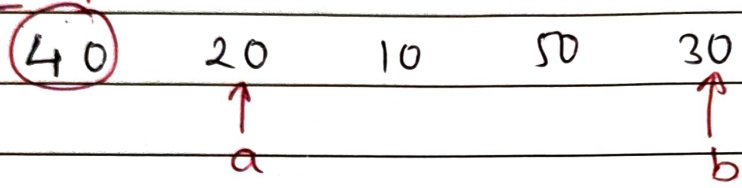
2. 'a' starts from the second element.
3. 'b' starts from the last element.
4. a moves from left to right.
5. b moves from right to left.
6. Move a while A[a] ≤ pivot.
7. move b while A[b] ≥ pivot.
8. If both a and b stop and a < b, swap A[a] and A[b].
9. After swapping, again move a and b according to rule 6 and 7.
10. If a ≥ b, the pointers have crossed:
    - ✓ stop moving a and b.
    - ✓ Do not swap a and b.
    - ✓ swap pivot with A[b].
11. The pivot is now in it's correct position.
12. Apply Quick sort seperately to:
    - left part
    - right part.
13. stop recursion when low >= high.

Example :-

40 20 10 50 30

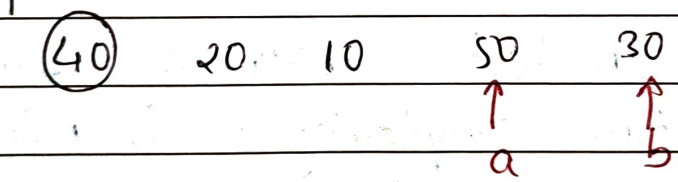
Let's trace Quick sort using the first element as pivot, with a = left pointer and b = right pointer.

Pass 1 :- p

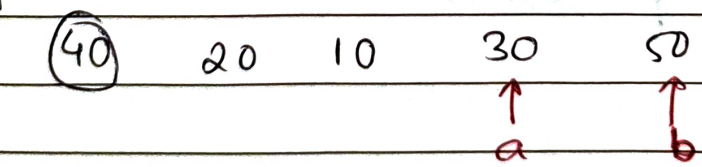


move a :-
- ✓ $20 \leq 40 \rightarrow$ a moves
- ✓ $10 \leq 40 \rightarrow$ a moves
- ✓ $50 > 40 \rightarrow$ a stops at 50

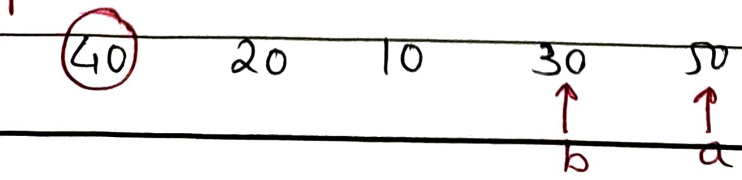
move b :-
- ✓ $30 \leq 40 \rightarrow$ b stops at 30



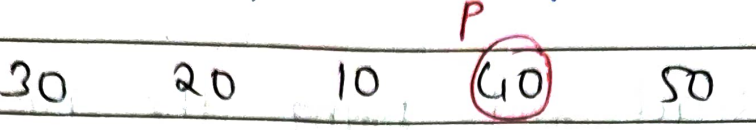
since, $a < b$, swap A[a] and A[B].



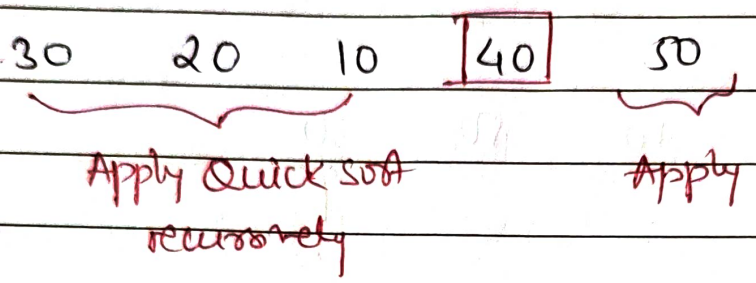
Again move a and b.
- ✓ a starts after 30 → reaches 50, because $50 > 40$ → stops.
- ✓ b moves left and reaches 30, because $30 \leq 40$ → stops.



Now, a has crossed b.
Therefore, stop and swap pivot (40) with A[b](30).



first partition completed.



Now quick sort is applied separately.
ie. left part and right part of pivot (40) element.

* Advantages :-

1) It is the fastest internal sorting algorithm with time complexity = $O(n \log n)$.
2) It is in-place algorithm and does not required additional memory to maintain another copy of data.

* Disadvantages :-

1) Worst case time complexity is $O(n^2)$.
2) Performance depends on the choice of pivot.
3) Recursive implementation may increase stack usage.

'C' program to implement Quick sort:

```
#include <stdio.h>
void quick_sort ( int arr[], int low, int high)
{   int pivot, a, b, temp;
    if (low < high)
    {   pivot = arr [low];      // first element as pivot
        a = low + 1;            // a = second element
        b = high;               // b = last element.
        while (a <= b)
        {                       // move to right
            while (a <= high && arr [a] <= pivot)
                a++;            // move to left.
            while (arr [b] > pivot)
                b--;
            if (a < b)          // if a & b have not crossed.
            {   temp = arr [a];
                arr [a] = arr [b];
                arr [b] = temp;
            }   }
        // If a and b crossed: swap pivot with b.
        temp = arr [low];
        arr [low] = arr [b];
        arr [b] = temp;
        // sort left part recursively.
        quick_sort ( arr, low, b-1);
        // sort right part recursively.
        quick_sort ( arr, b+1, high);
    }
}
```

Questions From DBATU Examination :

```
int main()
{
    int arr[50], n, i;
    printf ("enter number of elements");
    scanf ("%d", &n);
    printf ("enter elements");
    for (i=0; i<n; i++)
        scanf ("%d", &arr[i]);
    printf ("Before sort :- ");
    for (i=0; i<n; i++)
        printf ("%d\t", arr[i]);
    quick_sort (arr, 0, n-1);
    printf ("After sort :- ");
    for (i=0; i<n; i++)
        printf ("%d\t", arr[i]);
    return 0;
}
```

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Sorting : ⑤ Merge sort
2. ⑥ Heap sort.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

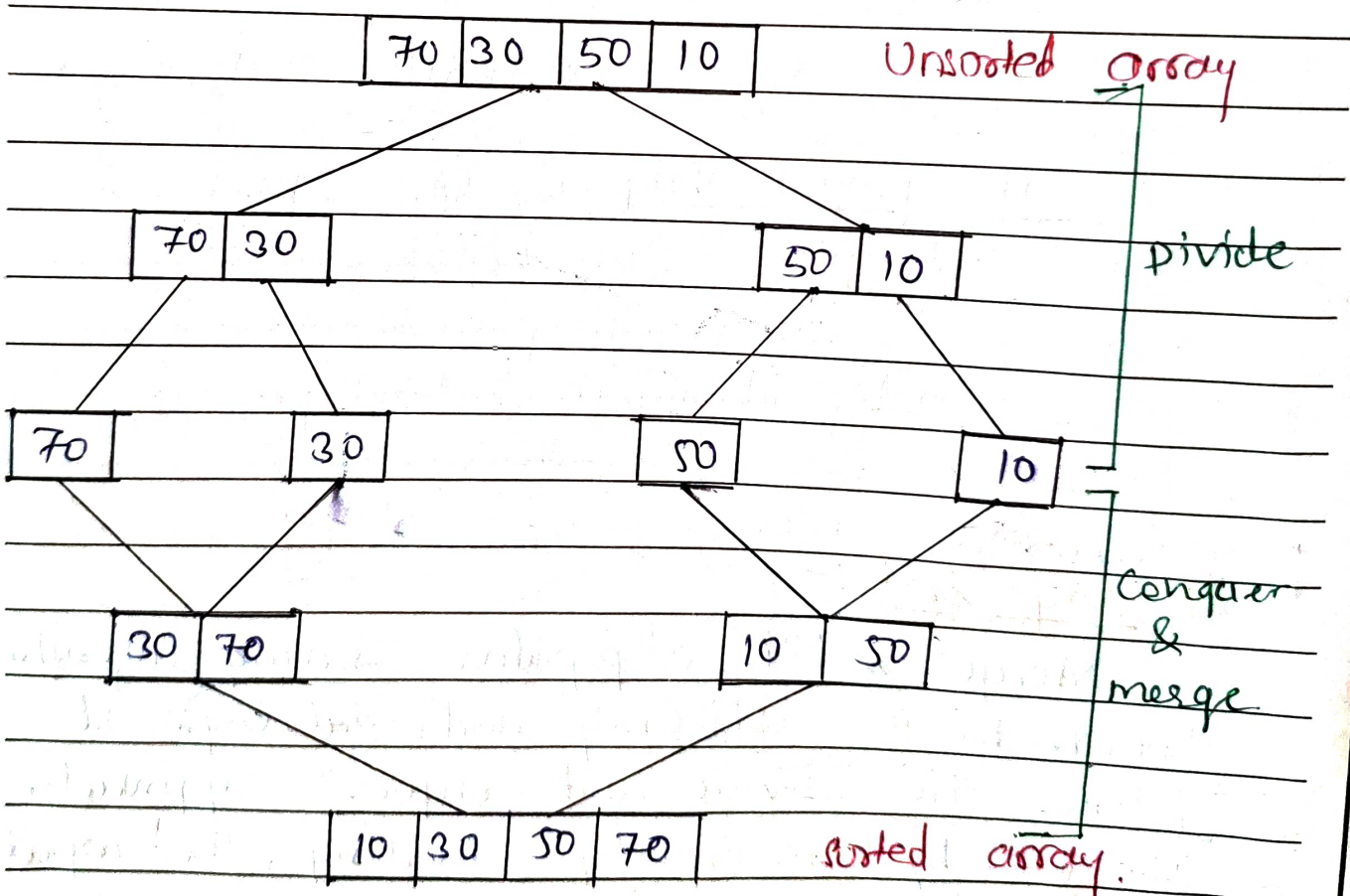
■ **Theory Content :**

Merge Sort :-

Merge sort is a popular sorting algorithm known for it's efficiency and stability. It follows the "Divide and conquer" approach. it works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Step by step explanation of how merge sort works -

1. Divide :- Divide the list or array recursively into two halves until it can no more be divided.
2. Conquer :- Each subarray is sorted individually using the merge sort algorithm.
3. Merge :- The sorted sub arrays ~~is~~ are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.



Example :- Array a[] = {38, 27, 43, 10}.

Divide :-

- ✓ [38, 27, 43, 10] is divided into [38, 27] and [43, 10].
- ✓ [38, 27] is divided into [38], [27].
- ✓ [43, 10] is divided into [43], [10].

Conquer :-

- ✓ [38] is already sorted. ✓ [27] is already sorted
- ✓ [27] is already sorted. ✓ [10] is already sorted.

Merge :-

- ✓ Merge [28] and [27] to get [27, 38].
- ✓ Merge [43] and [10] to get [10, 43].
- ✓ Merge [27, 38] and [10, 43] to get final sorted list [10, 27, 38, 43].

- Complexity Analysis :-

Time complexity for all the cases :- $O(n \log n)$.

- Advantages :-

1) Efficient for large data set
2) Stable sorting algorithm.
3) Suitable for external sorting.

- Disadvantages :-

1) Requires extra memory.
2) Recursive implementation can use additional stack space.

⑥ Heap Sort :-

Heap sort is a comparision-based sorting algorithm that uses a special binary tree called a Heap to sort elements.

It works in two main phases :-

① Build a Max Heap from the given array.

② Repeatedly remove the largest element (the root) and place it at the end of the array.

Heap sort is efficient because finding the largest element in a Max Heap takes $O(1)$ time, and restoring the heap after removal takes $O(\log n)$ time.

- **What is Heap ?**

A Heap is a "Complete Binary Tree" that follows a special rule called the Heap property.

A Complete Binary Tree Means -

✓ Every level is completely filled except possibly the last.

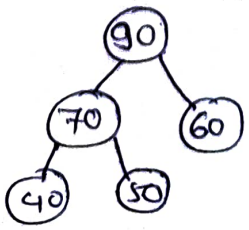
✓ The last level is filled from left to right.

There are two types of Heaps :-

(A) Max Heap :-

In a max heap, the parent node is always greater than or equal to its children.

Ex :-

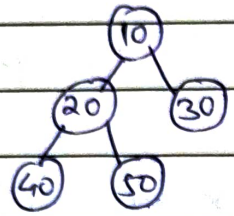


Largest element = 90 (Root).

(B) Min Heap :-

In a Min heap, the parent node is always smaller than or equal to it's children.

Ex :-



Smallest element = 10 (Root).

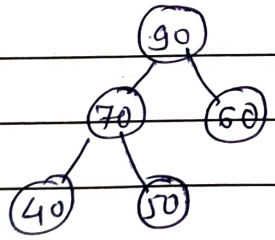
- Array Representation of Heap :-

A heap is usually sorted in an array.

Ex :-

Index :	0	1	2	3	4
Array :	90	70	60	40	50.

Tree Representation :-



- formula for parent & child :-

If the index of a node is "i" :

parent = $(i-1)/2$.

left child = $2i+1$

right child = $2i+2$. (using integer division).

- Algorithm :-

phase 1 : Build max heap.

phase 2 : Sorting.

1. Swap the root with last element.
2. Reduce the heap size by one.
3. Heapify the remaining elements
4. Repeat until only one element remains.

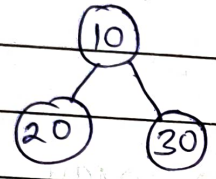
◎ Heapify :-

Heapify is the process of rearranging a node and it's children so that the heap property is satisfied.

✓ In a max Heap, the parent must be greater than or equal to it's children.

✓ In a min Heap, the parent must be smaller than or equal to it's children.

Ex:-

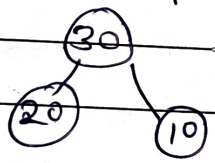


Here, parent = 10.
left child = 20
right child = 30.

This is not a max heap because 10 is smaller than both children.

Heapify :- Largest child = 30, swap 10 and 30

Result :-

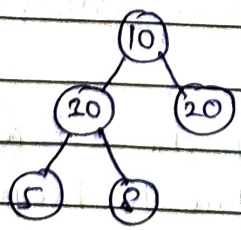


◎ Build max Heap :-

To convert the array into a max heap, we use the heapify process.

Ex:- Index : 0 1 2 3 4
Value : 10 30 20 5 8.

Tree ⇒



This is not a max heap because the root (10) is smaller than it's left child (30).

step 1: find the last non-leaf node.

formula:

Last non-leaf node = (n/2) - 1.

Here, n = 5.

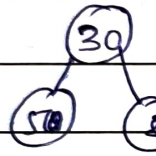
Last non-leaf node = (5/2) - 1

~~So, start Heapify~~ = 2 - 1 = 1.

So, start heapify from index 1.

step 2: Heapify index. 1.

Element = 30.

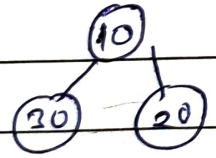


30 is already greater than both children. So, no swapping.

Array remains - 10 30 20 5 8.

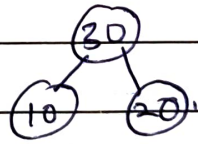
step 3: Heapify index 0.

Element = 10.



10 is less than 20 and 20. So swap 10 and 30

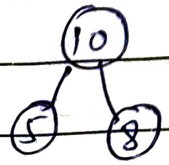
Tree after swapping -



Array becomes - 30 10 20 5 8.

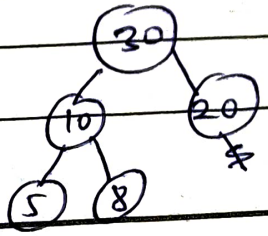
step 4: Heafify again.

Now we check the node where 10 moved (index 1).



10 is already greater than both children. so no swap needed.

final -



This satisfies the Max heap property because every parent is greater than or equal to its children.

+ Lecture No. : +

■ **Theory Content to be taught :**

1. (6) Heap sort – Continue...
2. performance and comparision among all the
3. methods.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

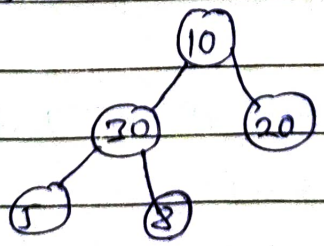
■ **Theory Content :**

Example :- 10 30 20 5 8. (max Heap).

Index = 0 1 2 3 4

Value = 10 30 20 5 8.

Array Representation –



This is not a max heap because 10 < 30.

phase 1: Build max heap.

step 1: find last non-leaf node.

Last non-leaf node = (n/2)-1

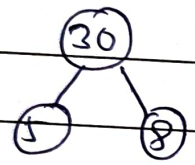
= (5/2)-1

= 1.

start heapifying from index 1.

step 2: Heapify index 1.

Node = 30.

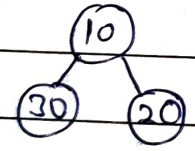


30 is greater than both children.
No swap required.

	0	1	2	3	4
Array remains -	10	30	20	5	8.

step 3: Heapify index 0.

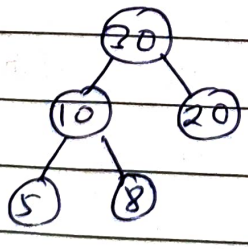
Node = 10.



10 is smaller than 30 and 20. Largest value is 30. So swap 10 and 30.

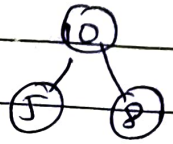
	0	1	2	3	4
Array after swap =	30	10	20	5	8

Tree -



step 4: Heapify index 1 again.

Node = 10.



10 is greater than both children.
No swap required.

	0	1	2	3	4
max heap =	30	10	20	5	8.

phase 2: Heap Sort.

	0	1	2	3	4
Max heap =	30	10	20	5	8

pass 1: swap root and last element.

swap (30, 8).

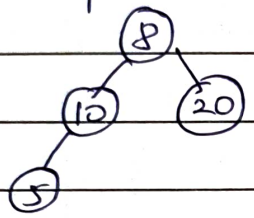
Array after swap = 8 10 20 5 [30]

30 is now in it's correct position.

Now, Heap Size = 4.

	0	1	2	3
Current Heap.	8	10	20	5

Tree =



Again apply phase 1 and phase 2 on above Tree. for the final result.

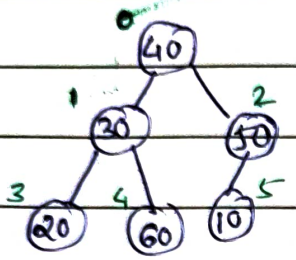
Performance and Comparison among all the methods.

Feature	Bubble sort	selection sort	Insertion sort	Quick sort	merge sort	Heap sort	
Best Time Complexity	$O(n)$	$O(n^2)$	$O(n)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	
Average Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	
Worst Time Complexity	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$	$O(n \log n)$	
Space Complexity	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$	$O(n)$	$O(1)$	
stable	yes	NO	yes	NO	yes	No.	
In-place	yes	yes	yes	yes	No	yes.	
suitable for large data.	No	NO	No	Yes	yes	yes.	
Technique used	swapping adjacent elements	Selecting minimum element	Inserting at correct position	Divide and Conquer	Divide and conquer	Heap (Binary Tree).	

• Example :- Heap sort in Ascending order using Max heap.

Elements — 40 30 50 20 60 10.

Step 1: Construct the initial binary tree.



Array Representation :-

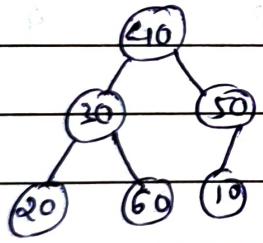
[40, 30, 50, 20, 60, 10].
0 1 2 3 4 5 ← index.

Step 2: Build Max heap.

Last non-leaf node = (n/2) - 1. (n = array size).

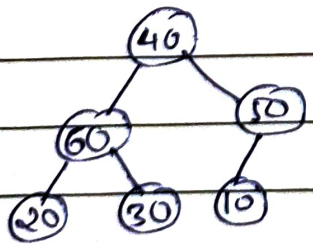
Heapify node 50 :

children = 10 → 50 is already greater.



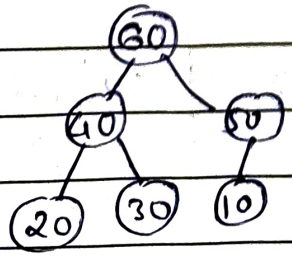
Heapify node 30 :

children = 20, 60 → largest is 60, so swap 30 and 60.



Heapify Root 40 :

Children = 60, 50 → largest is 60, so swap.

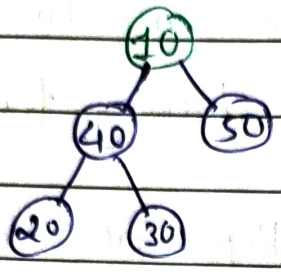


Now Max heap is :

[60, 40, 50, 20, 30, 10]
0 1 2 3 4 5 ← index.

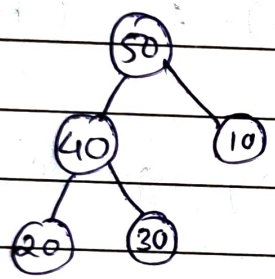
Step 3: Heap sort.

pass 1: swap root 60 with last element 10.



Sorted part = [60].

→ Again Heapify:

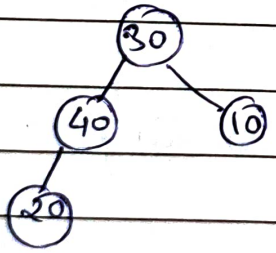


Array :-

50	40	10	20	30	60
0	1	2	3	4	sorted.

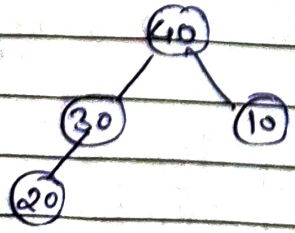
→ index.

pass 2: swap root 50 with 30.



Sorted part = [50, 60].

→ Again Heapify :-

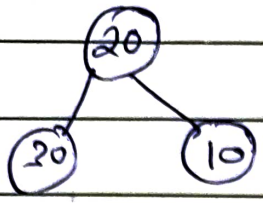


Array :-

40	30	10	20	50, 60
0	1	2	3	sorted.

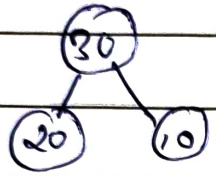
→ index.

pass 3 :- swap root 40 with 20.



sorted part = [40, 50, 60].

Again Heapify :-



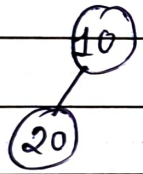
Array :-

[30, 20, 10, | 40, 50, 60]

0 1 2 sorted.

index

pass 4 :- swap root 30 with 10.



sorted part = [30, 40, 50, 60].

Again Heapify :-

Array :-

[20, 10 | 30, 40, 50, 60]

0 1 sorted.

index.

pass 5 :- swap root 20 with 10.

final array = [10, 20, 30, 40, 50, 60].

sorted.

Ascending order = 10, 20, 30, 40, 50, 60.

+ Lecture No. : +

■ **Theory Content to be taught :**

1. Hashing.
2.
3.
4.
5.

■ **Specific Objectives :**

1.
2.
3.
4.
5.

■ **Theory Content :**

◎ Hashing :-

Hashing is a technique used to store and retrieve data quickly. Instead of searching every element one by one, hashing calculates the location (index) where the data should be stored using a Hash Function. This allows searching, inserting, and deleting data in constant time (O(1)) on average.

Real Time example: Imagine a library with 1000 books.

If you want book number 820, you may have to check many books one by one. 1.... 820. (very slow)

With Hashing :- suppose every book is placed according to it's book number. Book 820 is directly stored at shelf 820. You can directly go to shelf 820. (very fast). This is exactly how hashing works.

- Basic Terms :-

A) Key :- The value used to identify data.

Ex :- roll_no = 25, empid = 108, phone_no = 123, etc.

Here 25, 108 are keys.

B) Hash function :- A mathematical formula that converts a key into an index.

Ex :- h(key) = key % table_size.

c) Hash Table :- A data structure used to store data according to the index produced by the hash function.

D) Hash value :- The index generated by the hash function.

Ex :- key = 25.

Hash function = 25 % 10 = 5. (10 = tablesize)

Hash value = 5.

Store 25 at index 5.

- How hashing Works?

Suppose, table size = 10

Hash function, h(key) = key % 10.

Insert 25, 36, 17, 48.

- Insert 25 ⇒

$h(25) = 25 \% 10 = 5$.

store at index 5.

index :	0	1	2	3	4	5	6	7	8	9
value :						25				

- Insert 36 ⇒

$h(36) = 36 \% 10 = 6$.

store at index 6.

index :	0	1	2	3	4	5	6	7	8	9
value :						25	36			

- Insert 17 ⇒

$h(17) = 17 \% 10 = 7$.

store at index 7.

index :	0	1	2	3	4	5	6	7	8	9
value :						25	36	17		

- Insert 48 ⇒

$h(48) = 48 \% 10 = 8$.

store at index 8.

index :	0	1	2	3	4	5	6	7	8	9
value :						25	36	17	48	

Searching becomes very fast.

Collision :-

Sometime two different keys produce the same hash value.
This is called Collision.

Ex :- Table size = 10.
keys are, 25 and 35.

Hash function = $h(25) = 25\%10 = 5$.
$h(35) = 35\%10 = 5$.

Both produce index 5.
This is called Collision.

Collision Resolution Techniques :-

There are two main methods:
1. separate chaining.
2. Open addressing.

1) separate chaining :- (open Hashing) :-

store multiple elements in a linked list at the same index.

2) Open addressing :- (closed Hashing) :-

If a position is occupied, search for another position.

Methods:
- ✓ Linear probing.
- ✓ Quadratic probing.
- ✓ Double Hashing.

- **Advantages:-**
  - ✓ very fast searching.
  - ✓ fast insertion
  - ✓ fast deletion.
  - ✓ Average time Complexity = O(1).

- **Disadvantages:-**
  - ✓ Collisions may occur.
  - ✓ choosing a poor hash function reduces performance.
  - ✓ Extra memory may be needed. (for separate chaining)

- **Comparison of Open Hashing & closed Hashing.**

parameter	Open Hashing	closed Hashing.
(a) storage location.	colliding elements are stored outside the hash table, usually in linked list.	All elements are stored inside the hash table itself.
(b) Memory Overhead.	Requires extra momerg for pointers/ linked list.	Requires less additional memory because no external linked list is needed.
(c) Deletion operation	Easier - simply remove the element from the linked list.	More complex / deletion may require special markers to maintain searching.
(d) Technique	Mainly separate chaining.	open addressing, such as Linear probing, Quadratic probing, & double hashing.